ΙΟΝΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ

ΦΟΙΤΗΤΕΣ:ΝΕΚΤΑΡΙΟΣ ΣΤΥΛΙΑΝΟΣ ΚΑΡΑΓΙΑΝΝΗΣ

ΚΑΘΗΓΗΤΕΣ:ΜΙΧΑΗΛ ΣΤΕΦΑΝΙΔΑΚΗΣ

ΜΑΘΗΜΑ:ΠΑΡΑΛΛΗΛΟΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

AM:Inf2021078

ΕΡΓΑΣΙΑ 1

06/04/2026

ΠΙΝΑΚΑΣ ΠΕΡΙΕΓΧΟΜΕΝΩΝ

Περιεχόμενα

ΠΙΝΑΚΑΣ ΠΕΡΙΕΓΧΟΜΕΝΩΝ 2
1.ΕΙΣΑΓΩΓΗ 3
2.ΣΕΙΡΙΑΚΗ ΕΠΙΛΥΣΗ 3
Ε.1ΣΕΙΡΙΑΚΗ ΕΠΙΛΥΣΗ 3
3.ΕΠΙΛΥΣΗ ΜΕ ΕΝΤΟΛΕΣ AVX 3
Ε.2 μετατροπή του σειριακού κώδικα σε κώδικα με εντολές AVX 3
Ε.3 Αξιοποίηση παραλληλίας 4
Ε.4 Επιβάρυνση 4
Ε.5 Χειρισμός στοιχείων 4
4.ΣΥΓΚΡΙΣΗ ΑΠΟΔΟΣΗΣ 4
5.ΑΝΑΦΟΡΑ ΣΤΗΝ ΧΡΗΣΗ ΤΝ 6

1.ΕΙΣΑΓΩΓΗ

Η παρούσα εργασία αφορά την υλοποίηση και βελτιστοποίηση ενός αλγορίθμου επεξεργασίας διδιάστατου πίνακα NxN τύπου float. Ο στόχος είναι ο υπολογισμός ενός πίνακα εξόδου B με βάση τα γειτονικά στοιχεία του πίνακα εισόδου A, αρχικά με σειριακή εκτέλεση και στη συνέχεια με χρήση AVX εντολών για παραλληλοποίηση.

Η εκφώνηση ορίζει ότι κάθε στοιχείο του πίνακα εξόδου υπολογίζεται ως γραμμικός συνδυασμός του ίδιου και των τεσσάρων γειτονικών στοιχείων του.

2.ΣΕΙΡΙΑΚΗ ΕΠΙΛΥΣΗ

Ε.1ΣΕΙΡΙΑΚΗ ΕΠΙΛΥΣΗ

Στη σειριακή υλοποίηση ο υπολογισμός εφαρμόζεται μόνο στα εσωτερικά στοιχεία του πίνακα, δηλαδή για δείκτες i=1 έως N-2 και j=1 έως N-2. Τα στοιχεία που βρίσκονται στα άκρα δεν υπολογίζονται, καθώς δεν διαθέτουν πλήρες σύνολο γειτονικών στοιχείων που απαιτεί ο τύπος

3.ΕΠΙΛΥΣΗ ΜΕ ΕΝΤΟΛΕΣ AVX

Ε.2 μετατροπή του σειριακού κώδικα σε κώδικα με εντολές AVX

Για τη μετατροπή του σειριακού κώδικα σε παραλληλοποιημένο με AVX επιλέχθηκε επεξεργασία δεδομένων σε blocks των 8 float στοιχείων. Οι AVX καταχωρητές τύπου __m256 επιτρέπουν την ταυτόχρονη επεξεργασία 8 στοιχείων.

Σε κάθε επανάληψη φορτώνονται:

- τα 8 κεντρικά στοιχεία
- τα 8 στοιχεία της πάνω γραμμής
- τα 8 στοιχεία της κάτω γραμμής

- τα 8 αριστερά γειτονικά
- τα 8 δεξιά γειτονικά

Στη συνέχεια εφαρμόζονται διανυσματικές πράξεις πολλαπλασιασμού και πρόσθεσης για τον υπολογισμό των αποτελεσμάτων.Η πρόσβαση στα δεδομένα βασίζεται στη μονοδιάστατη αποθήκευση του πίνακα σε row-major order.

Ε.3 Αξιοποίηση παραλληλίας

Σε κάθε πράξη AVX χρησιμοποιούνται και τα 8 στοιχεία του καταχωρητή, καθώς η επεξεργασία γίνεται σε blocks των 8 float. Έτσι επιτυγχάνεται πλήρης αξιοποίηση της διαθέσιμης παραλληλίας.

Ε.4 Επιβάρυνση

Η χρήση AVX εισάγει επιπλέον επιβάρυνση λόγω:

- πολλαπλών φορτώσεων δεδομένων από διαφορετικές θέσεις μνήμης
- προσβάσεων σε γειτονικά στοιχεία
- χρήσης unaligned loads για τα αριστερά και δεξιά στοιχεία

Επιπλέον, απαιτούνται περισσότερες πράξεις πρόσθεσης και πολλαπλασιασμού ανά επανάληψη.

Ε.5 Χειρισμός στοιχείων

Στην υλοποίηση με AVX ο υπολογισμός εφαρμόζεται μόνο στα εσωτερικά στοιχεία του πίνακα. Τα στοιχεία που βρίσκονται στα άκρα δεν υπολογίζονται και παραμένουν στις αρχικές τους τιμές, καθώς η επεξεργασία γίνεται σε blocks των 8 στοιχείων.

4.ΣΥΓΚΡΙΣΗ ΑΠΟΔΟΣΗΣ

Ε.6 Σύγκριση απόδοσης

N	SERIAL	AVX	SPEEDUP
96	0.000197	0.000134	1.4701
1000	0.022299	0.014239	1.5664
10000	4.992959	4.520750	1.1044

Η υλοποίηση με AVX παρουσιάζει σημαντική βελτίωση απόδοσης σε σχέση με τη σειριακή εκτέλεση. Για μικρές τιμές του N η διαφορά είναι μικρότερη, ενώ για

μεγαλύτερες τιμές η επιτάχυνση είναι πιο εμφανής. Το speedup δεν φτάνει θεωρητικά το 8x λόγω επιπλέον προσβάσεων μνήμης και υπολογιστικού overhead.

Από τις μετρήσεις προκύπτει ότι η υλοποίηση με AVX είναι ταχύτερη από τη σειριακή και στις τρεις περιπτώσεις. Για $N = 96$ το speedup είναι 1.47x για $N = 1000$είναι 1.57x ενώ για $N = 10000$ είναι 1.10x. Παρατηρείται ότι η βελτίωση απόδοσης υπάρχει, αλλά δεν προσεγγίζει το θεωρητικό 8x, καθώς η συγκεκριμένη μέθοδος απαιτεί πολλαπλές φορτώσεις από μνήμη και επιπλέον overhead λόγω πρόσβασης σε γειτονικά στοιχεία.

ΣΕΙΡΙΑΚΗ

```
Nek_PC4@DESKTOP-1KDAEHQ UCRT64 /H/OneDrive/Υπολογιστής/παρραλληλος_προγραμματισμος
$ gcc -Wall -O2 SERIAL.c -o serial -DN=96 -DR=100

Nek_PC4@DESKTOP-1KDAEHQ UCRT64 /H/OneDrive/Υπολογιστής/παρραλληλος_προγραμματισμος
$ ./serial
sum = 40720780.000000
 check = 40720780.000000
time = 0.000197 sec

Nek_PC4@DESKTOP-1KDAEHQ UCRT64 /H/OneDrive/Υπολογιστής/παρραλληλος_προγραμματισμος
$ gcc -Wall -O2 SERIAL.c -o serial -DN=1000 -DR=100

Nek_PC4@DESKTOP-1KDAEHQ UCRT64 /H/OneDrive/Υπολογιστής/παρραλληλος_προγραμματισμος
$ ./serial
sum = 497930764288.000000
 check = 497930764288.000000
time = 0.022299 sec

Nek_PC4@DESKTOP-1KDAEHQ UCRT64 /H/OneDrive/Υπολογιστής/παρραλληλος_προγραμματισμος
$ gcc -Wall -O2 SERIAL.c -o serial -DN=10000 -DR=100

Nek_PC4@DESKTOP-1KDAEHQ UCRT64 /H/OneDrive/Υπολογιστής/παρραλληλος_προγραμματισμος
$ ./serial
sum = 2251799813685248.000000
 check = 2251799813685248.000000
time = 4.992959 sec
```

AVX

```
Nek_PC4@DESKTOP-1KDAEHQ UCRT64 /H/OneDrive/Υπολογιστής/παρραλληλος_προγραμματισμος
$ gcc -Wall -O2 -mavx AVX-C.c -o avx -DN=10000 -DR=100

Nek_PC4@DESKTOP-1KDAEHQ UCRT64 /H/OneDrive/Υπολογιστής/παρραλληλος_προγραμματισμος
$ ./avx
check = 2251799813685248.000000
time = 4.520750 sec

Nek_PC4@DESKTOP-1KDAEHQ UCRT64 /H/OneDrive/Υπολογιστής/παρραλληλος_προγραμματισμος
$ gcc -Wall -O2 -mavx AVX-C.c -o avx -DN=1000 -DR=100

Nek_PC4@DESKTOP-1KDAEHQ UCRT64 /H/OneDrive/Υπολογιστής/παρραλληλος_προγραμματισμος
$ ./avx
check = 497930764288.000000
time = 0.014239 sec

Nek_PC4@DESKTOP-1KDAEHQ UCRT64 /H/OneDrive/Υπολογιστής/παρραλληλος_προγραμματισμος
$ gcc -Wall -O2 -mavx AVX-C.c -o avx -DN=96 -DR=100

Nek_PC4@DESKTOP-1KDAEHQ UCRT64 /H/OneDrive/Υπολογιστής/παρραλληλος_προγραμματισμος
$ ./avx
check = 40720780.000000
time = 0.000134 sec

Nek_PC4@DESKTOP-1KDAEHQ UCRT64 /H/OneDrive/Υπολογιστής/παρραλληλος_προγραμματισμος
$
```

5.ΑΝΑΦΟΡΑ ΣΤΗΝ ΧΡΗΣΗ ΤΝ

Κατά την εκπόνηση της εργασίας χρησιμοποιήθηκαν εργαλεία τεχνητής νοημοσύνης για:

- κατανόηση της εκφώνησης
- σχεδιασμό της λύσης με AVX
- υποστήριξη στη συγγραφή του κώδικα
- βοήθεια στη διατύπωση της αναφοράς

Το όνομα της πλατφόρμας είναι Open AI και χρησιμοποιήθηκε κατά κύριο λόγο κατανόησης MSYS2 UCRT64

Η τελική υλοποίηση, οι επιλογές σχεδιασμού και ο έλεγχος των αποτελεσμάτων πραγματοποιήθηκαν από τον φοιτητή.